

TWO POINTERS TECHNIQUE

1. Overview

The Two Pointers technique is a fundamental algorithmic pattern where two pointers (indices, references, or iterators) traverse a data structure — typically an array, string, or linked list — to solve problems efficiently. Instead of using nested loops ($O(n^2)$ or worse), two pointers often reduce complexity to $O(n)$ by processing elements in a single pass.

The two pointers can move in the same direction, opposite directions, or at different speeds. The key insight is that each pointer helps the other avoid redundant work by maintaining a window, a constraint, or a comparison that shrinks the search space.

This technique appears in nearly every SDE placement drive, online assessment, and competitive programming contest. It is one of the most asked topics in coding interviews at companies like Google, Amazon, Microsoft, Meta, and Uber.

2. Intuition

Simple Explanation

Imagine you have a long line of numbered boxes and you need to find a pair that sums to a target. With brute force, you'd check every pair — box 1 with box 2, box 1 with box 3, ... box 1 with box N, then box 2 with box 3, and so on. That's $O(n^2)$.

With two pointers, you place one at the start and one at the end. If the sum is too small, you move the start forward. If too large, you move the end backward. Each step eliminates one element from consideration. You find the answer in $O(n)$ without ever checking all pairs.

Analogy

Tug of war: Two people stand at opposite ends of a rope. They walk toward each other. At each step, the person on the left (who sees the smaller number) and the person on the right (who sees the larger number) decide who moves. Together, they find the pair they're looking for.

Fast and slow runners: On a circular track, if one runner is faster than the other, the faster runner will eventually lap the slower one. This is exactly how we detect cycles in a linked list.

Step-by-Step Reasoning

1. **Identify** that the problem involves a linear data structure (array, string, linked list)
2. **Recognize** that a brute force nested loop would solve it, but you need better
3. **Place** two pointers based on the problem's constraints:
 - Opposite ends → for sorted arrays, pairs, palindromes
 - Same start → for subarrays, windows, duplicate removal
 - Different speeds → for cycle detection, middle element
4. **Move** pointers according to a condition that guarantees progress

5. **Stop** when pointers meet, cross, or reach the end

Why It Works

Two pointers work because they exploit **structure** in the data — sorted order, monotonicity, or a fixed window property. Each pointer carries information about a region of the data, and by moving strategically, we eliminate large portions of the search space without examining them explicitly.

3. When to Use It

Use Two Pointers when:

Scenario	Example
Array is sorted (or can be sorted)	Pair sum, 3Sum, 4Sum
Need to find a pair/triplet/quadruplet	Two Sum II, 3Sum, 4Sum
Need to remove duplicates in-place	Remove Duplicates from Sorted Array
Need to partition or rearrange	Dutch National Flag, Sort Colors
Need to find a subarray/substring	Container With Most Water
Need to detect a cycle	Linked List Cycle
Need to check palindrome	Valid Palindrome
Need to find middle element	Middle of Linked List
Need to compare from both ends	Palindrome, Two Sum
Need to merge two sorted arrays	Merge Sorted Array
Linear data structure involved	Array, String, Linked List

Trigger Phrases

- "sorted array"
- "pair whose sum"
- "in-place"
- "without extra space"
- "remove duplicates"
- "O(1) extra space"
- "linked list cycle"
- "palindrome"
- "container with most water"
- "trapping rain water"
- "sort colors"
- "triplet sum"
- "quadruplet sum"
- "middle of linked list"

- "merge two sorted arrays"
- "subarray with given sum"

4. When Not to Use It

Situation	Reason	Better Alternative
Unsorted array, need arbitrary pair	Sorting breaks original indices	HashMap (Two Sum)
Need all pairs, not just existence	Two pointers finds one or a subset	Nested loops or hash map
Multiple queries on same array	Preprocess once, answer many	Prefix sums, hash map
Data structure is not linear	Two pointers only works on sequences	Depends on structure
Need to find a subarray with sum k (with negatives)	Two pointers fails when monotonicity is broken	Prefix sum + hash map
Need random access but only have iterator	Some linked list problems need fast access	Hash set, tortoise-hare
Array is dynamic (insertions/deletions)	Pointers shift, indices invalidate	Balanced BST, segment tree
Problem is about counting, not finding	Two pointers may miss some counts	Sliding window, combinatorics

Common Wrong Assumptions

- ❌ "Two pointers always work on sorted arrays" → Only if the problem involves a monotonic property
- ❌ "Two pointers is always $O(n)$ " → Can be $O(n^2)$ if you restart pointers (e.g., naive 3Sum)
- ❌ "Two pointers works on unsorted arrays without sorting" → Rarely; usually need sorted order
- ❌ "Fast and slow pointers always detect cycles" → Only if you handle null checks properly

5. Core Concepts

5.1 Opposite Direction Pointers

What it means: Two pointers start at opposite ends of the array and move toward each other.

Why it matters: This is the most common two-pointer pattern. It leverages sorted order to eliminate elements from consideration efficiently.

Example: Finding a pair that sums to target in a sorted array. Left pointer at 0, right pointer at $n-1$. If $\text{sum} < \text{target}$, move left forward. If $\text{sum} > \text{target}$, move right backward.

5.2 Same Direction Pointers

What it means: Both pointers start at the same end (usually left) and move forward, but at different rates or with different conditions.

Why it matters: Used for in-place removal, window problems, and partition-based algorithms.

Example: Removing duplicates from sorted array. Slow pointer tracks the position to write, fast pointer scans for new elements.

5.3 Fast and Slow Pointers (Tortoise and Hare)

What it means: One pointer moves twice as fast as the other. Used primarily in linked lists.

Why it matters: Detects cycles in $O(n)$ time and $O(1)$ space without modifying the data structure.

Example: Linked List Cycle. Fast moves 2 steps, slow moves 1 step. If they meet, there's a cycle.

5.4 Window Maintenance

What it means: Pointers define a sliding window (subarray/substring) that expands or contracts based on conditions.

Why it matters: Solves subarray/substring problems in $O(n)$ instead of $O(n^2)$.

Example: Container With Most Water. Left and right pointers define the container width. The shorter wall determines the height.

5.5 Partitioning

What it means: Using multiple pointers to partition an array into 2 or 3 regions.

Why it matters: Enables in-place sorting and partitioning with $O(1)$ extra space.

Example: Dutch National Flag problem. Three pointers partition the array into three regions: 0s, 1s, and 2s.

6. Step-by-Step Algorithm (General Framework)

Opposite Direction Pattern

1. Initialize `left = 0`, `right = n - 1`
2. While `left < right`:
 - a. Compute current value based on `arr[left]` and `arr[right]`
 - b. If current value matches target, return/record
 - c. If current value `<` target, move left forward (`left++`)
 - d. If current value `>` target, move right backward (`right--`)
3. If no solution found, return default (`null`, `-1`, `false`, etc.)

Same Direction Pattern

1. Initialize `slow = 0, fast = 0` (or `fast = 1`)
2. While `fast < n`:
 - a. Compare `arr[fast]` with `arr[slow]` or some condition
 - b. If condition met, update `slow` position and write
 - c. Move `fast` forward
3. Return `slow + 1` (length of processed region) or the result

Fast and Slow Pattern

1. Initialize `slow = head, fast = head`
2. While `fast != null` and `fast.next != null`:
 - a. Move `slow` one step: `slow = slow.next`
 - b. Move `fast` two steps: `fast = fast.next.next`
 - c. If `slow == fast`, cycle detected, return `true`
3. Return `false` (no cycle)

7. Dry Run

Problem: Two Sum II (Sorted Array)

Input: `arr = [2, 7, 11, 15]`, `target = 9`

Step	left	right	arr[left]	arr[right]	sum	Action
1	0	3	2	15	17	sum > target → right--
2	0	2	2	11	13	sum > target → right--
3	0	1	2	7	9	sum == target → Found!

Result: indices `[0, 1]` → values `[2, 7]`

Problem: Remove Duplicates from Sorted Array

Input: `arr = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`

Step	slow	fast	arr[slow]	arr[fast]	Action
1	0	1	0	0	Equal → move fast
2	0	2	0	1	Different → slow++, arr[1]=1, fast++
3	1	3	1	1	Equal → move fast
4	1	4	1	1	Equal → move fast
5	1	5	1	2	Different → slow++, arr[2]=2, fast++
6	2	6	2	2	Equal → move fast

Step	slow	fast	arr[slow]	arr[fast]	Action
7	2	7	2	3	Different → slow++, arr[3]=3, fast++
8	3	8	3	3	Equal → move fast
9	3	9	3	4	Different → slow++, arr[4]=4, fast++

Result: slow = 4, length = 5, array = [0, 1, 2, 3, 4, ...]

Problem: Container With Most Water

Input: height = [1, 8, 6, 2, 5, 4, 8, 3, 7]

Step	left	right	width	minH	area	maxArea	Action
1	0	8	8	1	8	8	height[left] < height[right] → left++
2	1	8	7	7	49	49	height[left] > height[right] → right--
3	1	7	6	3	18	49	height[left] > height[right] → right--
4	1	6	5	8	40	49	height[left] == height[right] → left++
5	2	6	4	6	24	49	height[left] < height[right] → left++
6	3	6	3	2	6	49	height[left] < height[right] → left++
7	4	6	2	5	10	49	height[left] < height[right] → left++
8	5	6	1	4	4	49	left == right → stop

Result: maxArea = 49

Problem: Linked List Cycle Detection

Input: 3 → 2 → 0 → -4 → (back to 2)

Step	slow	fast	slow.val	fast.val	Condition
1	3	3	3	3	Initial
2	2	0	2	0	No match
3	0	2	0	2	No match
4	-4	-4	-4	-4	Match! Cycle detected

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// =====
```

```
// 1. OPPOSITE DIRECTION POINTERS – Two Sum II (Sorted Array)
// =====
// Returns indices (1-based) of the two numbers that sum to target
vector<int> twoSumSorted(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;

    while (left < right) {
        int sum = nums[left] + nums[right];

        if (sum == target) {
            return {left + 1, right + 1}; // 1-based indices
        } else if (sum < target) {
            left++; // Need larger sum
        } else {
            right--; // Need smaller sum
        }
    }

    return {}; // No solution found
}

// =====
// 2. SAME DIRECTION POINTERS – Remove Duplicates
// =====
// Returns the length of array after removing duplicates in-place
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;

    int slow = 0; // Points to last unique element

    for (int fast = 1; fast < nums.size(); fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }

    return slow + 1; // Length of unique elements
}

// =====
// 3. CONTAINER WITH MOST WATER
// =====
int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int maxWater = 0;

    while (left < right) {
        int width = right - left;
        int minHeight = min(height[left], height[right]);
        int currentWater = width * minHeight;
        maxWater = max(maxWater, currentWater);

        // Move the pointer with the smaller height
    }
}
```

```

        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return maxWater;
}

// =====
// 4. TRAPPING RAIN WATER
// =====
int trap(vector<int>& height) {
    if (height.empty()) return 0;

    int left = 0, right = height.size() - 1;
    int leftMax = 0, rightMax = 0;
    int totalWater = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) {
                leftMax = height[left];
            } else {
                totalWater += leftMax - height[left];
            }
            left++;
        } else {
            if (height[right] >= rightMax) {
                rightMax = height[right];
            } else {
                totalWater += rightMax - height[right];
            }
            right--;
        }
    }

    return totalWater;
}

// =====
// 5. DUTCH NATIONAL FLAG (Sort Colors)
// =====
void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;

    while (mid <= high) {
        switch (nums[mid]) {
            case 0: // Red
                swap(nums[low], nums[mid]);
                low++;
                mid++;
                break;

```

```

        case 1: // White
            mid++;
            break;
        case 2: // Blue
            swap(nums[mid], nums[high]);
            high--;
            break;
    }
}

// =====
// 6. 3SUM
// =====
vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>> result;
    int n = nums.size();
    if (n < 3) return result;

    sort(nums.begin(), nums.end());

    for (int i = 0; i < n - 2; i++) {
        // Skip duplicate values for the first element
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = n - 1;
        int target = -nums[i];

        while (left < right) {
            int sum = nums[left] + nums[right];

            if (sum == target) {
                result.push_back({nums[i], nums[left], nums[right]});

                // Skip duplicates for left and right
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;

                left++;
                right--;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
    }

    return result;
}

// =====
// 7. 4SUM
// =====

```

```

vector<vector<int>> fourSum(vector<int>& nums, int target) {
    vector<vector<int>> result;
    int n = nums.size();
    if (n < 4) return result;

    sort(nums.begin(), nums.end());

    for (int i = 0; i < n - 3; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        for (int j = i + 1; j < n - 2; j++) {
            if (j > i + 1 && nums[j] == nums[j - 1]) continue;

            int left = j + 1, right = n - 1;
            long long remaining = (long long)target - nums[i] - nums[j];

            while (left < right) {
                long long sum = (long long)nums[left] + nums[right];

                if (sum == remaining) {
                    result.push_back({nums[i], nums[j], nums[left], nums[right]});

                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;

                    left++;
                    right--;
                } else if (sum < remaining) {
                    left++;
                } else {
                    right--;
                }
            }
        }
    }

    return result;
}

// =====
// 8. LINKED LIST CYCLE (Fast and Slow Pointers)
// =====
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

bool hasCycle(ListNode* head) {
    if (!head || !head->next) return false;

    ListNode* slow = head;
    ListNode* fast = head;

```

```

    while (fast && fast->next) {
        slow = slow->next;          // Move 1 step
        fast = fast->next->next;     // Move 2 steps

        if (slow == fast) return true; // Cycle detected
    }

    return false; // No cycle
}

// =====
// 9. PALINDROME CHECKING
// =====
bool isPalindrome(string s) {
    // Remove non-alphanumeric and convert to lowercase
    int left = 0, right = s.length() - 1;

    while (left < right) {
        // Skip non-alphanumeric characters
        while (left < right && !isalnum(s[left])) left++;
        while (left < right && !isalnum(s[right])) right--;

        if (tolower(s[left]) != tolower(s[right])) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}

// =====
// 10. LINKED LIST PALINDROME CHECKING
// =====
bool isPalindromeLinkedList(ListNode* head) {
    if (!head || !head->next) return true;

    // Step 1: Find middle using slow and fast pointers
    ListNode* slow = head;
    ListNode* fast = head;

    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Step 2: Reverse the second half
    ListNode* prev = nullptr;
    ListNode* curr = slow->next;
    while (curr) {
        ListNode* nextNode = curr->next;

```

```

        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    // Step 3: Compare first half and reversed second half
    ListNode* first = head;
    ListNode* second = prev;
    while (second) {
        if (first->val != second->val) return false;
        first = first->next;
        second = second->next;
    }

    return true;
}

// =====
// EXAMPLE USAGE
// =====
int main() {
    // Two Sum
    vector<int> arr1 = {2, 7, 11, 15};
    auto res = twoSumSorted(arr1, 9);
    cout << "Two Sum: [" << res[0] << ", " << res[1] << "]\n";

    // Remove Duplicates
    vector<int> arr2 = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
    int len = removeDuplicates(arr2);
    cout << "After removing duplicates, length = " << len << "\n";

    // Container With Most Water
    vector<int> heights = {1, 8, 6, 2, 5, 4, 8, 3, 7};
    cout << "Max water area = " << maxArea(heights) << "\n";

    // Trapping Rain Water
    vector<int> elevation = {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1};
    cout << "Total trapped water = " << trap(elevation) << "\n";

    // Sort Colors
    vector<int> colors = {2, 0, 2, 1, 1, 0};
    sortColors(colors);
    cout << "Sorted colors: ";
    for (int c : colors) cout << c << " ";
    cout << "\n";

    // 3Sum
    vector<int> arr3 = {-1, 0, 1, 2, -1, -4};
    auto triplets = threeSum(arr3);
    cout << "3Sum triplets:\n";
    for (auto& t : triplets) {
        cout << " [" << t[0] << ", " << t[1] << ", " << t[2] << "]\n";
    }
}

```

```

// 4Sum
vector<int> arr4 = {1, 0, -1, 0, -2, 2};
auto quads = fourSum(arr4, 0);
cout << "4Sum quadruplets:\n";
for (auto& q : quads) {
    cout << " [" << q[0] << ", " << q[1] << ", " << q[2] << ", " << q[3] <<
"]\n";
}

// Palindrome String
string str = "A man, a plan, a canal: Panama";
cout << "Is palindrome: " << (isPalindrome(str) ? "Yes" : "No") << "\n";

return 0;
}

```

9. Python Implementation

```

from typing import List, Optional

# =====
# 1. OPPOSITE DIRECTION POINTERS – Two Sum II (Sorted Array)
# =====
def two_sum_sorted(nums: List[int], target: int) -> List[int]:
    """Returns 1-based indices of two numbers that sum to target."""
    left, right = 0, len(nums) - 1

    while left < right:
        current_sum = nums[left] + nums[right]

        if current_sum == target:
            return [left + 1, right + 1]
        elif current_sum < target:
            left += 1
        else:
            right -= 1

    return []

# =====
# 2. SAME DIRECTION POINTERS – Remove Duplicates
# =====
def remove_duplicates(nums: List[int]) -> int:
    """Removes duplicates in-place, returns length of unique elements."""
    if not nums:
        return 0

    slow = 0 # Points to last unique element

    for fast in range(1, len(nums)):

```

```

        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]

    return slow + 1

# =====
# 3. CONTAINER WITH MOST WATER
# =====
def max_area(height: List[int]) -> int:
    """Finds the maximum area of water a container can hold."""
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        min_height = min(height[left], height[right])
        current_water = width * min_height
        max_water = max(max_water, current_water)

        # Move the pointer with the smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water

# =====
# 4. TRAPPING RAIN WATER
# =====
def trap(height: List[int]) -> int:
    """Calculates total water trapped between bars."""
    if not height:
        return 0

    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    total_water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                total_water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                total_water += right_max - height[right]
            right -= 1

```

```

    return total_water

# =====
# 5. DUTCH NATIONAL FLAG (Sort Colors)
# =====
def sort_colors(nums: List[int]) -> None:
    """Sorts array with 0, 1, 2 in-place (Dutch National Flag)."""
    low, mid, high = 0, 0, len(nums) - 1

    while mid <= high:
        if nums[mid] == 0: # Red
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1: # White
            mid += 1
        else: # Blue
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1

# =====
# 6. 3SUM
# =====
def three_sum(nums: List[int]) -> List[List[int]]:
    """Finds all unique triplets that sum to zero."""
    result = []
    n = len(nums)
    if n < 3:
        return result

    nums.sort()

    for i in range(n - 2):
        # Skip duplicate values
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        target = -nums[i]

        while left < right:
            current_sum = nums[left] + nums[right]

            if current_sum == target:
                result.append([nums[i], nums[left], nums[right]])

                # Skip duplicates
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1

            left += 1
            right -= 1

```

```

        elif current_sum < target:
            left += 1
        else:
            right -= 1

    return result

# =====
# 7. 4SUM
# =====
def four_sum(nums: List[int], target: int) -> List[List[int]]:
    """Finds all unique quadruplets that sum to target."""
    result = []
    n = len(nums)
    if n < 4:
        return result

    nums.sort()

    for i in range(n - 3):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        for j in range(i + 1, n - 2):
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue

            left, right = j + 1, n - 1
            remaining = target - nums[i] - nums[j]

            while left < right:
                current_sum = nums[left] + nums[right]

                if current_sum == remaining:
                    result.append([nums[i], nums[j], nums[left], nums[right]])

                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    while left < right and nums[right] == nums[right - 1]:
                        right -= 1

                    left += 1
                    right -= 1
                elif current_sum < remaining:
                    left += 1
                else:
                    right -= 1

            return result

# =====
# 8. LINKED LIST CYCLE (Fast and Slow Pointers)
# =====
class ListNode:

```

```

def __init__(self, x):
    self.val = x
    self.next = None

def has_cycle(head: Optional[ListNode]) -> bool:
    """Detects cycle in linked list using Floyd's algorithm."""
    if not head or not head.next:
        return False

    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False

# =====
# 9. PALINDROME CHECKING
# =====
def is_palindrome(s: str) -> bool:
    """Checks if string is a palindrome (alphanumeric only)."""
    left, right = 0, len(s) - 1

    while left < right:
        # Skip non-alphanumeric characters
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True

# =====
# 10. LINKED LIST PALINDROME CHECKING
# =====
def is_palindrome_linked_list(head: Optional[ListNode]) -> bool:
    """Checks if linked list is a palindrome."""
    if not head or not head.next:
        return True

    # Step 1: Find middle using slow and fast pointers
    slow = head
    fast = head

```

```

while fast.next and fast.next.next:
    slow = slow.next
    fast = fast.next.next

# Step 2: Reverse the second half
prev = None
curr = slow.next
while curr:
    next_node = curr.next
    curr.next = prev
    prev = curr
    curr = next_node

# Step 3: Compare first half and reversed second half
first = head
second = prev
while second:
    if first.val != second.val:
        return False
    first = first.next
    second = second.next

return True

# =====
# EXAMPLE USAGE
# =====
if __name__ == "__main__":
    # Two Sum
    print(f"Two Sum: {two_sum_sorted([2, 7, 11, 15], 9)}")

    # Remove Duplicates
    arr = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
    length = remove_duplicates(arr)
    print(f"After removing duplicates, length = {length}")

    # Container With Most Water
    print(f"Max water area = {max_area([1, 8, 6, 2, 5, 4, 8, 3, 7])}")

    # Trapping Rain Water
    print(f"Total trapped water = {trap([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1])}")

    # Sort Colors
    colors = [2, 0, 2, 1, 1, 0]
    sort_colors(colors)
    print(f"Sorted colors: {colors}")

    # 3Sum
    print(f"3Sum triplets: {three_sum([-1, 0, 1, 2, -1, -4])}")

    # 4Sum
    print(f"4Sum quadruplets: {four_sum([1, 0, -1, 0, -2, 2], 0)}")

```

```
# Palindrome String
print(f"Is palindrome: {is_palindrome('A man, a plan, a canal: Panama')}")
```

10. Code Explanation

Opposite Direction: Two Sum II

Why it works: The array is sorted. When the sum is too small, we need a larger number → move left forward. When sum is too large, we need a smaller number → move right backward. This eliminates one element per step, guaranteeing $O(n)$.

Edge cases handled: If no solution exists, returns empty vector. The loop condition `left < right` ensures we don't use the same element twice.

Same Direction: Remove Duplicates

Why it works: `slow` marks the position where the next unique element should be written. `fast` scans for the next distinct value. Since the array is sorted, duplicates are adjacent. When `nums[fast] != nums[slow]`, we've found a new unique element.

Edge cases handled: Empty array returns 0 immediately. Single element array works because the loop doesn't execute.

Container With Most Water

Why it works: The area is `width × min(height[left], height[right])`. The limiting factor is the shorter wall. Moving the taller wall inward would only decrease the width without increasing the height (since the shorter wall still limits it). So we always move the pointer with the smaller height.

Edge cases handled: Two-element array works naturally. Equal heights are handled by the else branch.

3Sum

Why it works: Fix one element, then use two-pointer on the remaining subarray. Sorting allows us to skip duplicates easily. The outer loop fixes `nums[i]`, and the inner two-pointer finds pairs that sum to `-nums[i]`.

Duplicate handling: Three levels of dedup:

1. Skip duplicate `nums[i]` values
2. Skip duplicate `nums[left]` after a match
3. Skip duplicate `nums[right]` after a match

Fast and Slow Pointers (Cycle Detection)

Why it works: If there's a cycle, the fast pointer (2 steps) will eventually catch up to the slow pointer (1 step) from behind. The relative speed is 1 step per iteration, so the distance between them shrinks by 1 each time.

Edge cases handled: Null head, single node without cycle, single node pointing to itself (cycle).

Palindrome Checking

Why it works: Compare characters from both ends moving inward. The first mismatch means it's not a palindrome. Skip non-alphanumeric characters to handle strings like "A man, a plan, a canal: Panama".

Edge cases handled: Empty string, single character, strings with only non-alphanumeric characters.

11. Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Best Case	Worst Case
Two Sum II (Sorted)	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Remove Duplicates	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Container With Most Water	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Trapping Rain Water	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Dutch National Flag	$O(n)$	$O(1)$	$O(n)$	$O(n)$
3Sum	$O(n^2)$	$O(1)$ or $O(n)$ for sorting	$O(n^2)$	$O(n^2)$
4Sum	$O(n^3)$	$O(1)$ or $O(n)$ for sorting	$O(n^3)$	$O(n^3)$
Linked List Cycle	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Palindrome (String)	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Palindrome (Linked List)	$O(n)$	$O(1)$	$O(n)$	$O(n)$

Notes:

- Sorting costs $O(n \log n)$ for 3Sum and 4Sum but is typically not counted separately in the overall complexity (the $O(n^2)$ or $O(n^3)$ dominates)
- 3Sum can be optimized to $O(n^2)$ using two pointers after sorting
- 4Sum can be optimized to $O(n^3)$ using two nested loops + two pointers
- Space is $O(1)$ for all algorithms except the sorting overhead

12. Common Patterns

Pattern 1: Pair Sum / Target Sum

How to identify: "Find two numbers that sum to target", "pair with given sum", "two sum"

General approach: Sort the array (if not already sorted), use opposite direction pointers. If $\text{sum} < \text{target}$, move left; if $\text{sum} > \text{target}$, move right.

Example problems: Two Sum II (LeetCode 167), Pair with Given Sum (GFG)

Pattern 2: In-Place Removal

How to identify: "Remove duplicates", "remove element", "in-place", " $O(1)$ extra space"

General approach: Use same direction pointers. **slow** tracks write position, **fast** scans for the next valid element.

Example problems: Remove Duplicates from Sorted Array (LeetCode 26), Remove Element (LeetCode 27)

Pattern 3: Container / Area Problems

How to identify: "Container with most water", "maximum area", "largest rectangle"

General approach: Opposite direction pointers. The constraint is usually the shorter side. Move the pointer that is the limiting factor.

Example problems: Container With Most Water (LeetCode 11), Trapping Rain Water (LeetCode 42)

Pattern 4: 3Sum / K-Sum Family

How to identify: "Triplet sum", "find three numbers", "k-sum", "3Sum", "4Sum"

General approach: Sort the array. Fix (k-2) elements using nested loops, use two-pointer for the remaining two. Handle duplicates at every level.

Example problems: 3Sum (LeetCode 15), 4Sum (LeetCode 18), 3Sum Closest (LeetCode 16)

Pattern 5: Partition / Segregation

How to identify: "Sort colors", "Dutch national flag", "segregate", "partition array into three"

General approach: Use 3 pointers (low, mid, high) to partition the array into 3 regions. One pointer for each boundary.

Example problems: Sort Colors (LeetCode 75), Segregate 0s and 1s (GFG)

Pattern 6: Cycle Detection

How to identify: "Linked list cycle", "detect cycle", "find duplicate number", "happy number"

General approach: Fast and slow pointers. If they meet, cycle exists. To find the cycle start, reset one pointer to head and move both at same speed.

Example problems: Linked List Cycle (LeetCode 141), Linked List Cycle II (LeetCode 142), Find the Duplicate Number (LeetCode 287)

Pattern 7: Palindrome Checking

How to identify: "Palindrome", "valid palindrome", "symmetric"

General approach: Two pointers from opposite ends, compare characters. Skip non-alphanumeric if needed.

Example problems: Valid Palindrome (LeetCode 125), Palindrome Linked List (LeetCode 234), Valid Palindrome II (LeetCode 680)

Pattern 8: Middle Element / Finding the Midpoint

How to identify: "Middle of linked list", "find median", "find midpoint"

General approach: Fast and slow pointers. When fast reaches the end, slow is at the middle.

Example problems: Middle of the Linked List (LeetCode 876), Reorder List (LeetCode 143)

13. Common Mistakes

Off-by-One Errors

- ✗ Using `left <= right` instead of `left < right` in opposite direction → can cause using the same element twice
- ✗ Off-by-one in duplicate skipping: `while (left < right && nums[left] == nums[left+1]) left++;` — correctly advances past the last duplicate
- ✗ Forgetting that `fast.next` could be null when checking `fast.next.next` in cycle detection

Wrong Initialization

- ✗ `slow = 0, fast = 0` for remove duplicates → need to start fast at 1 to compare with slow
- ✗ `slow = head, fast = head->next` for cycle detection → works but needs different logic; `fast = head` is simpler
- ✗ Forgetting to sort before 3Sum/4Sum

Boundary Conditions

- ✗ Not checking `nums.empty()` before accessing `nums[0]`
- ✗ Not checking `head == nullptr` or `head->next == nullptr` in linked list problems
- ✗ 3Sum: not checking `n < 3` before the loop

Overflow

- ✗ Using `int` for sum in 4Sum when values can be large → use `long long`
- ✗ `width * height` in Container With Most Water could overflow for large arrays

Duplicate Handling

- ✗ Not skipping duplicates in 3Sum/4Sum → results contain duplicate triplets
- ✗ Wrong duplicate skip logic: checking `nums[i] == nums[i+1]` instead of `nums[i] == nums[i-1]` → skips valid triplets
- ✗ Forgetting to skip duplicates for both `left` and `right` after a match

Pointer Movement

- ✗ Moving both pointers after every comparison in opposite direction → may miss the solution
- ✗ Not incrementing `mid` when swapping with `high` in Dutch National Flag
- ✗ Moving `slow` at wrong time in remove duplicates

Misunderstanding the Algorithm

- ✗ Thinking 3Sum can be solved in $O(n)$ — it's $O(n^2)$ with two pointers

- ✗ Using two pointers on unsorted array without sorting
- ✗ Applying opposite direction pointers to find subarray with given sum containing negative numbers

14. Edge Cases

General Edge Cases

Edge Case	What to Check
Empty array	Return 0, empty, or false immediately
Single element	Loop condition handles it, or check separately
Two elements	Minimum for pair-based algorithms
All elements equal	Duplicate handling, palindrome, cycle detection
Already sorted	Works fine, just verify
Reverse sorted	Works for opposite direction, verify sorted order
All negative values	Sum logic, target comparison
Large values	Overflow (use long long)
No solution exists	Return default value
Duplicate values	Dedup logic in 3Sum/4Sum

Specific Edge Cases

Two Sum II:

- Target smaller than any pair sum → pointers will cross without finding
- Target larger than any pair sum → same
- Multiple valid pairs → first found is returned

Remove Duplicates:

- Array with no duplicates → slow moves at same rate as fast
- All same elements → slow stays at 0
- Single element → returns 1

Container With Most Water:

- Two elements → one calculation, done
- Decreasing heights → area decreases as pointers move inward
- All equal heights → area depends only on width

Trapping Rain Water:

- Strictly increasing → no water trapped
- Strictly decreasing → no water trapped
- Plateau (all equal) → no water trapped

- Single tall bar in middle → water on both sides

Dutch National Flag:

- All same color → one pass, no swaps
- Two colors only → still works
- Already sorted → no unnecessary swaps

3Sum/4Sum:

- Less than 3/4 elements → empty result
- All zeros → one triplet [0,0,0]
- All same values → no triplet (unless 0+0+0=0)
- Large negative values → potential overflow

Linked List Cycle:

- No cycle → fast reaches null
- Cycle at head → fast catches slow immediately
- Single node pointing to itself → cycle detected
- Two nodes forming cycle → fast catches slow

Palindrome:

- Empty string → true
 - Single character → true
 - " " (space only) → true (after skipping non-alphanumeric)
 - Case sensitivity → handle with tolower/toupper
 - String with only non-alphanumeric → true
-

15. Variations

15.1 Three Pointers (Dutch National Flag)

What changes: Instead of two pointers, we use three (low, mid, high) to partition an array into three regions.

When used: Whenever you need to segregate three distinct values (e.g., 0, 1, 2 in Sort Colors).

Importance: High for placements. Directly asked in Sort Colors (LeetCode 75).

15.2 K-Sum Generalization

What changes: For k-sum, fix (k-2) elements with nested loops, use two-pointer for the last two. Generalizes to k-sum in $O(n^{k-1})$.

When used: 3Sum, 4Sum, 5Sum, etc.

Importance: High. 3Sum and 4Sum are extremely common.

15.3 Two Pointers with HashMap

What changes: First pass creates a hash map (value → index), then a single pointer checks for complement.

When used: Unsorted arrays where sorting is not allowed (need original indices).

Importance: High. Two Sum (LeetCode 1) is the most classic example.

15.4 Sliding Window (Substring/Subarray)

What changes: Two pointers define a window that expands (right moves) and contracts (left moves) based on a condition.

When used: Problems like "smallest subarray with sum $\geq K$ ", "longest substring without repeating characters".

Importance: Very high. A separate topic but closely related to two pointers.

15.5 Fast and Slow Variations

Find middle: **slow** moves 1 step, **fast** moves 2 steps. When **fast** reaches end, **slow** is at middle.

Find cycle start: After detecting cycle, reset one pointer to head, move both at same speed. Meeting point is cycle start.

Find duplicate in array: Treat array values as "next pointer" in a linked list, use fast and slow pointers.

Importance: High. Cycle detection and middle element are frequently asked.

15.6 Palindrome Variations

Valid Palindrome II: Can delete at most one character. Use two pointers, when mismatch occurs, check two substrings (skip left or skip right).

Palindrome Linked List: Use fast/slow to find middle, reverse second half, compare.

Importance: Medium. Appears in interviews but less frequently than core palindrome.

15.7 Two Pointers on Multiple Arrays

What changes: Each pointer is on a different array. Merge two sorted arrays, find intersection of two arrays.

When used: Merging, intersection, union of sorted arrays.

Importance: Medium. Merge Sorted Array (LeetCode 88) is a classic.

16. Related Algorithms / Data Structures

Two Pointers vs Sliding Window

Aspect	Two Pointers	Sliding Window
Direction	Often opposite, can be same	Always same direction
Window	May not track a window explicitly	Explicitly maintains a window
Use case	Pair finding, partition, comparison	Subarray/substring problems

Aspect	Two Pointers	Sliding Window
Example	Two Sum II, 3Sum	Longest substring without repeating characters

Choose Two Pointers when: The problem involves comparing elements from opposite ends, finding pairs, or partitioning.

Choose Sliding Window when: The problem asks for a contiguous subarray/substring that satisfies a condition.

Two Pointers vs Binary Search

Aspect	Two Pointers	Binary Search
Search space	Eliminates one element per step	Eliminates half the elements per step
When to use	Multiple elements interact	Finding a single element
Example	Pair sum, palindrome	Search in sorted array

Choose Two Pointers when: You need to compare or combine two elements.

Choose Binary Search when: You need to find a single element in a sorted/monotonic space.

Two Pointers vs Hashing

Aspect	Two Pointers	Hashing
Space	$O(1)$	$O(n)$
Time	$O(n)$ after sorting	$O(n)$
Preserves original indices	No (sorting changes order)	Yes
Example	Two Sum II (sorted)	Two Sum (unsorted)

Choose Two Pointers when: Space is constrained, or array is already sorted.

Choose Hashing when: Array is unsorted and you need $O(n)$ time with $O(n)$ space.

Two Pointers vs Floyd's Cycle Detection

These are the same thing! Fast and slow pointers **is** Floyd's Cycle Detection algorithm. The same technique is used for:

- Cycle detection in linked lists
- Finding the middle of a linked list
- Finding the duplicate number in an array

Two Pointers vs Partitioning Algorithms

The Dutch National Flag algorithm is a specific application of two pointers (actually three pointers) for partitioning. QuickSort's partition function also uses two pointers. The key difference is in the number of regions:

- QuickSort partition: 2 regions ($\leq$ pivot, $>$ pivot)
- Dutch National Flag: 3 regions (0, 1, 2)

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Two Sum II — Input Array Is Sorted	LeetCode 167	Opposite direction pointers, find pair summing to target	Easy
Valid Palindrome	LeetCode 125	Two pointers from ends, skip non-alphanumeric	Easy
Remove Duplicates from Sorted Array	LeetCode 26	Same direction pointers, in-place removal	Easy
Middle of the Linked List	LeetCode 876	Fast and slow pointers	Easy
Merge Sorted Array	LeetCode 88	Two pointers from ends of two arrays	Easy
Squares of a Sorted Array	LeetCode 977	Opposite direction pointers, compare absolute values	Easy

Medium

Problem	Platform	Main Idea	Difficulty
3Sum	LeetCode 15	Fix one element, two-pointer for remaining two	Medium
Container With Most Water	LeetCode 11	Opposite direction, move shorter wall	Medium
Sort Colors	LeetCode 75	Dutch National Flag, three pointers	Medium
Remove Duplicates from Sorted Array II	LeetCode 80	Same direction, allow at most 2 duplicates	Medium
4Sum	LeetCode 18	Fix two elements, two-pointer for remaining two	Medium
Trapping Rain Water	LeetCode 42	Two pointers with left/right max tracking	Medium
Linked List Cycle II	LeetCode 142	Find cycle start using Floyd's algorithm	Medium
Palindrome Linked List	LeetCode 234	Fast/slow to find middle, reverse, compare	Medium

Problem	Platform	Main Idea	Difficulty
3Sum Closest	LeetCode 16	Two-pointer with closest tracking	Medium

Valid Palindrome II	LeetCode 680	Two-pointer, allow one deletion	Medium
----------------------------	--------------	---------------------------------	--------

Hard

Problem	Platform	Main Idea	Difficulty
---------	----------	-----------	------------

Trapping Rain Water II	LeetCode 407	3D version, priority queue + BFS	Hard
-------------------------------	--------------	----------------------------------	------

Find the Duplicate Number	LeetCode 287	Fast and slow on array as linked list	Medium*
----------------------------------	--------------	---------------------------------------	---------

Longest Valid Parentheses	LeetCode 32	Two passes with two pointers	Hard
----------------------------------	-------------	------------------------------	------

Minimum Window Substring	LeetCode 76	Two pointers + hash map (sliding window)	Hard
---------------------------------	-------------	--	------

Subarrays with K Different Integers	LeetCode 992	Two pointers + counting	Hard
--	--------------	-------------------------	------

4Sum II	LeetCode 454	Four arrays, hash map + two-pointer	Medium
----------------	--------------	-------------------------------------	--------

Candy	LeetCode 135	Two passes with two pointers	Hard
--------------	--------------	------------------------------	------

* Find the Duplicate Number is classified as Medium on LeetCode but is Hard conceptually.

18. Interview Explanation

Here's what to say when asked about Two Pointers in an interview:

"Two Pointers is a technique where we use two reference points — usually indices in an array or nodes in a linked list — to traverse the data structure in a single pass, avoiding nested loops.

The core idea is that by placing pointers strategically and moving them based on some condition, we eliminate elements from consideration without explicitly checking them. This typically reduces time complexity from $O(n^2)$ to $O(n)$.

There are three main patterns:

1. Opposite direction: Both pointers start at the ends and move toward each other. This is used when the array is sorted and we need to find a pair or check a condition from both ends. Classic example is Two Sum on a sorted array — if the sum is too small, move the left pointer forward; if too large, move the right pointer backward.

2. Same direction: Both pointers start at the same end and move forward, one slower than the other. This is used for in-place removal, like removing duplicates from a sorted array. The slow pointer marks where to write, and the fast pointer scans for new values.

3. Fast and slow pointers: One pointer moves twice as fast as the other. This is used in linked lists to detect cycles — if there's a cycle, the fast pointer will eventually lap the slow one. It's also used to find the middle of a linked list.

The key insight is that each pointer carries information about a region of the data, and by moving strategically, we avoid redundant work. The space complexity is almost always $O(1)$ since we're just using two index variables.

Common mistakes to watch for: off-by-one in the loop condition (should be $\text{left} < \text{right}$, not $\text{left} \leq \text{right}$), forgetting to handle duplicates in 3Sum and 4Sum, and not checking for null pointers in linked list problems."

19. Revision Notes

Opposite Direction Pointers

- **When:** Sorted array, need pair/comparison from both ends
- **Template:**

```
left = 0, right = n-1
while (left < right):
    process(arr[left], arr[right])
    if condition: left++
    else: right--
```

- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Off-by-one (use $<$ not $\leq$), unsorted array won't work

Same Direction Pointers

- **When:** In-place removal, partition, duplicate elimination
- **Template:**

```
slow = 0
for (fast = 1; fast < n; fast++):
    if (arr[fast] != arr[slow]):
        slow++
        arr[slow] = arr[fast]
return slow + 1
```

- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Wrong initialization ($\text{slow}=0$, $\text{fast}=1$), not handling empty array

Fast and Slow Pointers

- **When:** Cycle detection, middle element, duplicate in array
- **Template (cycle):**

```
slow = fast = head
while (fast && fast.next):
    slow = slow.next
    fast = fast.next.next
    if (slow == fast) return true
return false
```

- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Null check for `fast.next` before `fast.next.next`

3Sum / 4Sum

- **Pattern:** Sort + fix (k-2) elements + two-pointer for remaining two
- **Duplicate handling:** Skip at every level (`if (i > 0 && nums[i] == nums[i-1]) continue`)
- **Complexity:** 3Sum = $O(n^2)$, 4Sum = $O(n^3)$
- **Traps:** Overflow in 4Sum (use `long long`), forgetting to skip duplicates

Dutch National Flag

- **Pattern:** low=0, mid=0, high=n-1
- **Rules:**
 - `nums[mid] == 0` → swap with low, low++, mid++
 - `nums[mid] == 1` → mid++
 - `nums[mid] == 2` → swap with high, high--
- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Not incrementing `mid` when swapping with `high`

Container With Most Water

- **Pattern:** Move the pointer with smaller height
- **Formula:** `area = min(height[l], height[r]) × (r - l)`
- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Wrong direction (moving taller wall instead of shorter)

Trapping Rain Water

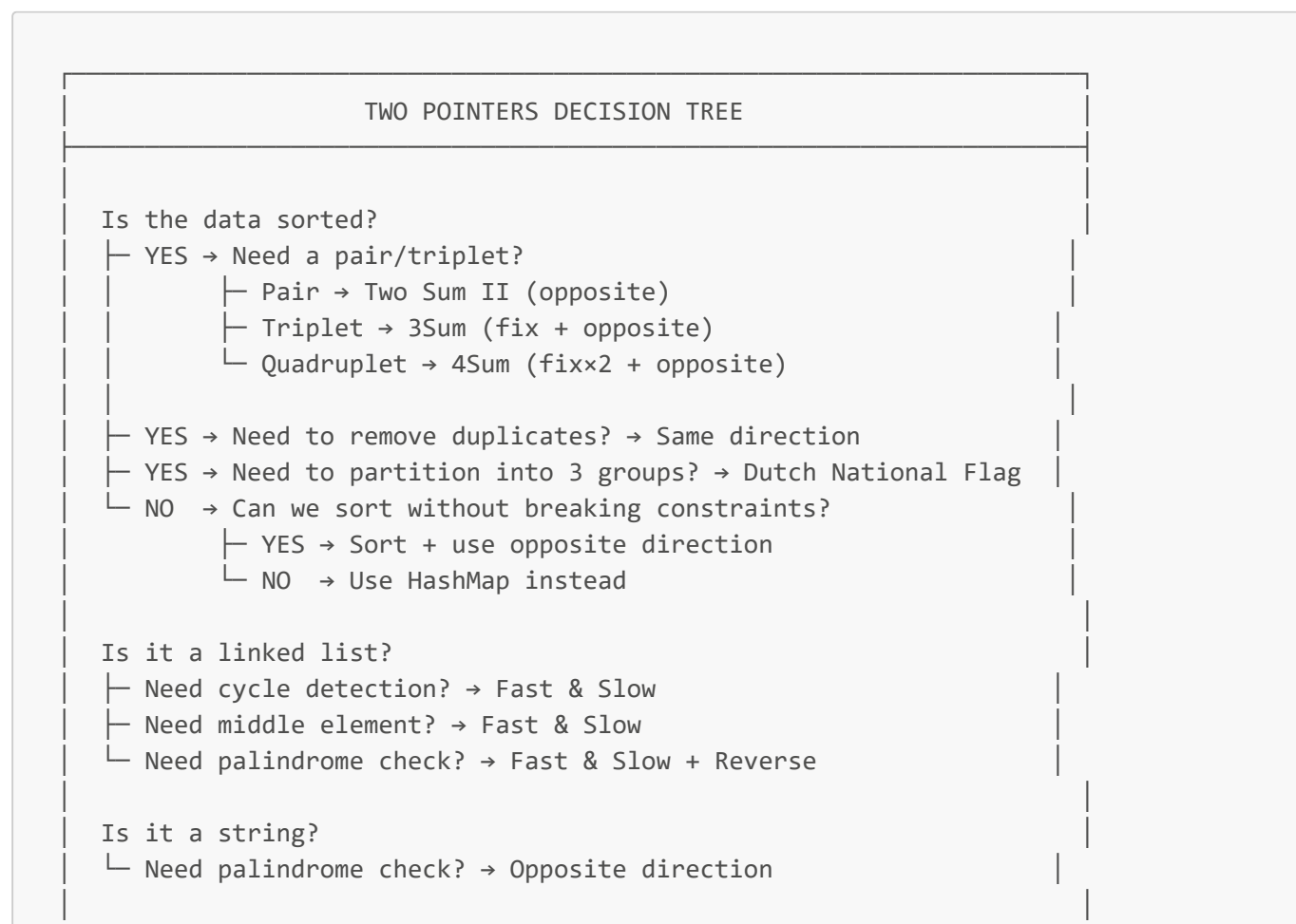
- **Pattern:** Two pointers tracking leftMax and rightMax
- **Key insight:** Water at position `i` = `min(leftMax, rightMax) - height[i]`
- **Complexity:** $O(n)$ time, $O(1)$ space
- **Traps:** Confusing with Container With Most Water

20. Final Cheat Sheet

Two Pointers — Quick Reference

Algorithm	Pattern	Time	Space	Key Condition
Two Sum II	Opposite	$O(n)$	$O(1)$	$\text{Sum} < \text{target} \rightarrow \text{left}++; \text{else right}--$
Remove Duplicates	Same	$O(n)$	$O(1)$	$\text{nums}[\text{fast}] \neq \text{nums}[\text{slow}] \rightarrow \text{write}$
Container With Most Water	Opposite	$O(n)$	$O(1)$	Move shorter wall
Trapping Rain Water	Opposite	$O(n)$	$O(1)$	Track leftMax, rightMax
Dutch National Flag	3-Pointer	$O(n)$	$O(1)$	0→swap low, 1→mid++, 2→swap high
3Sum	Opposite (nested)	$O(n^2)$	$O(1)$	Fix one, two-pointer rest
4Sum	Opposite (nested×2)	$O(n^3)$	$O(1)$	Fix two, two-pointer rest
Linked List Cycle	Fast & Slow	$O(n)$	$O(1)$	$\text{slow} == \text{fast} \rightarrow \text{cycle}$
Palindrome (String)	Opposite	$O(n)$	$O(1)$	$\text{s}[\text{left}] \neq \text{s}[\text{right}] \rightarrow \text{false}$
Palindrome (LL)	Fast & Slow + Reverse	$O(n)$	$O(1)$	Find middle, reverse, compare

When to Use What



```
Area/Water problem?
├ Container (max area between two lines) → Opposite, move shorter
└ Trapping rain water → Opposite with max tracking
```

Key Code Snippets

```
// Opposite direction – pair sum
while (l < r) {
    int s = a[l] + a[r];
    if (s == t) return {l+1, r+1};
    (s < t) ? l++ : r--;
}

// Same direction – remove duplicates
int s = 0;
for (int f = 1; f < n; f++)
    if (a[f] != a[s]) a[++s] = a[f];
return s + 1;

// Fast & slow – cycle detection
while (f && f->next) {
    s = s->next; f = f->next->next;
    if (s == f) return true;
}

// Dutch National Flag
while (m <= h) {
    if (a[m] == 0) swap(a[l++], a[m++]);
    else if (a[m] == 1) m++;
    else swap(a[m], a[h--]);
}

// 3Sum pattern
sort(a.begin(), a.end());
for (int i = 0; i < n-2; i++) {
    if (i > 0 && a[i] == a[i-1]) continue;
    int l = i+1, r = n-1, t = -a[i];
    while (l < r) {
        int s = a[l] + a[r];
        if (s == t) { /* record */ l++; r--; while(l<r && a[l]==a[l-1]) l++;
while(l<r && a[r]==a[r+1]) r--; }
        else if (s < t) l++;
        else r--;
    }
}
```

Must-Remember Numbers

Problem	LeetCode	Difficulty	Key Technique
Two Sum II	167	Easy	Opposite direction
Remove Duplicates	26	Easy	Same direction
Valid Palindrome	125	Easy	Opposite direction
Container With Most Water	11	Medium	Opposite, move shorter
3Sum	15	Medium	Fix + opposite
Sort Colors	75	Medium	3 pointers
Trapping Rain Water	42	Hard	Opposite with max tracking
Linked List Cycle	141	Easy	Fast & slow
Palindrome Linked List	234	Medium	Fast & slow + reverse
4Sum	18	Medium	Fix $\times 2$ + opposite

Most Common Trap Checklist

- ☐ `left < right` not `left <= right`
- ☐ Sorted array required for opposite direction
- ☐ Skip duplicates in 3Sum/4Sum at every level
- ☐ Use `long long` for large sums in 4Sum
- ☐ Check `fast && fast->next` for linked list
- ☐ Move pointer with **smaller** height in Container
- ☐ Don't increment `mid` when swapping with `high` in Dutch Flag
- ☐ Handle empty and single-element inputs
- ☐ Case-insensitive comparison in palindrome
- ☐ Skip non-alphanumeric characters in palindrome